

TD - DartWars

Initiation au langage Dart



Consignes.....	2
01 - Variables, valeurs et types.....	4
02 - Console / Conditions.....	5
03 - Boucles et listes.....	6
04 - POO + package tiers + tests unitaires.....	7
05 - Enum, Boucles, random.....	9
06 - Fonctions + tests unitaires.....	11
07 - Asynchronisme.....	12
Crédits.....	13

Consignes

- Consultez la **documentation du langage Dart** pour réaliser les exercices (<https://dart.dev/guides>). L'auto-documentation fait partie de l'exercice.
- **Il ne s'agit pas d'un exercice de vitesse mais de familiarisation au langage Dart.**

Prenez le temps de consulter la documentation du langage et de tester différentes alternatives pour parvenir au résultat visé. **Pour la qualité de votre apprentissage, un usage modéré et raisonné de l'IA générative est recommandé.**

- A l'aide du *CLI* de Dart, depuis votre terminal de commande, créez un **nouveau projet Dart** nommé ***dartwars*** (ne pas créer de projet Flutter).
Avec votre terminal de commande, placez-vous à la racine du dossier *dartwars* créé à la saisie de la commande précédente.
- Pour vérifier son bon fonctionnement, exécutez le programme puis ses tests unitaires

```
$ dart create dartwars
$ cd dartwars
$ dart run
Building package executable...
Built demo:demo.
Hello world: 42!

$ dart test
Building package executable... (1.7s)
Built test:test.
00:00 +1: All tests passed!
```

- Pour chaque exercice, créez un nouveau fichier dans le dossier ***./lib*** (ex: *./lib/exercice01.dart*).

Insérez votre code dans une fonction principale nommée ***run***. Pour afficher les valeurs de votre programme, employez l'instruction ***print()***.

Fichier : *lib/exercice01.dart*

```
void run() {
  print("hello world");
}
```

- Importez chaque fichier d'exercice (ex: ./lib/exercice01.dart) dans le fichier ./bin/dartwars.dart afin d'exécuter la fonction **run** de ce fichier.

Fichier : bin/dartwars.dart

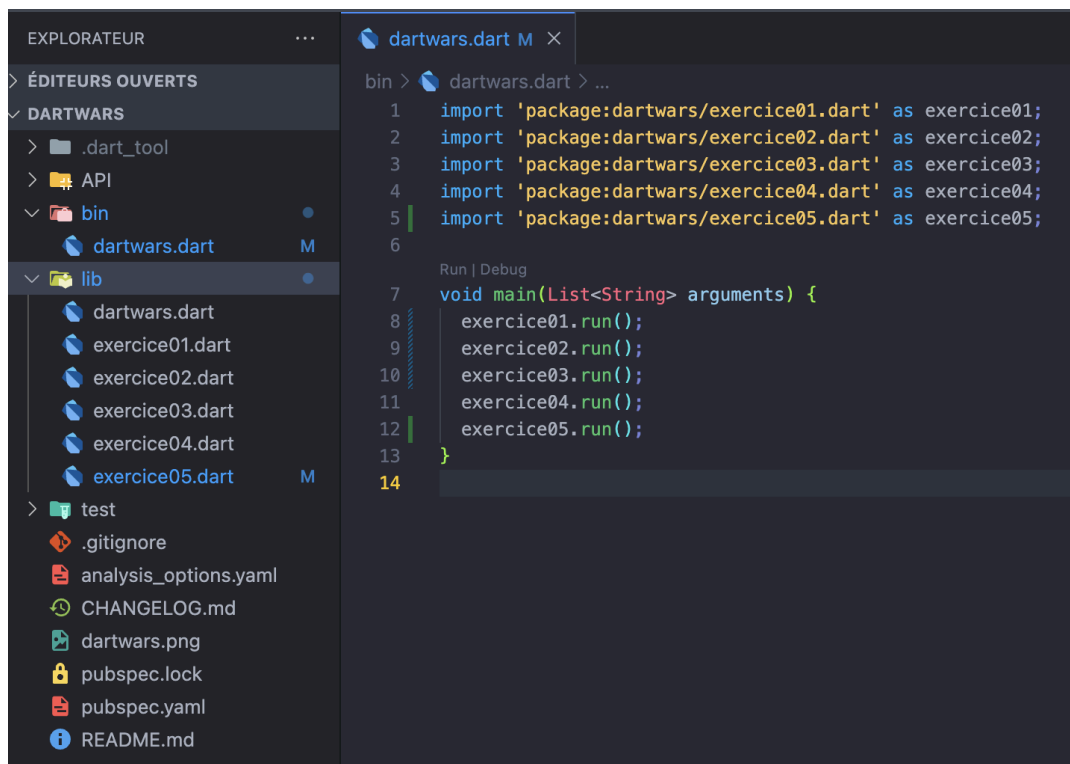
```
import 'package:dartwars/exercice01.dart' as exercice01;

void main(List<String> arguments) {
  exercice01.run();
}
```

Exécutez votre programme avec la commande \$ **dart run**

Exécutez les tests unitaires avec la commande \$ **dart test**

Pour vérifier les assertions, exécutez votre programme en mode debug.



01 - Variables, valeurs et types

- Créez un fichier `./lib/exercice01.dart`
- Déclarez les variables ***name***, ***height***, ***mass***, ***birthYear*** et ***isHuman***. Affectez-leur le bon type afin de pouvoir contenir la valeur fournie en exemple ci-dessous.
- Affichez ces informations dans le terminal de commande à l'aide de la méthode ***print***.

```
name : C-3PO  
height : 167  
mass : 75  
birth year : 112BBY  
isHuman : false
```

[Exercice 02 page suivante](#)

02 - Console / Conditions

- Créez un fichier `./lib/exercice02.dart`
- Importez la bibliothèque `dart:io` afin d'interagir avec l'utilisateur via le terminal de commande.
- Implémentez un programme proposant à l'utilisateur de choisir son camp en saisissant une valeur dans le terminal de commande :
 - en tapant 1, il rejoint la Rébellion,
 - en tapant 2, il rejoint l'Empire,
 - s'il tape un autre caractère, un message d'erreur *"Bad choice!"* est affiché.
- En fonction de la sélection de l'utilisateur, affichez un message de bienvenue adapté.

```
Select your side (1 - Rebels / 2 - Empire) :  
1  
Welcome to the Rebels ! May the Force be with you!  
  
2  
Welcome to the Dark Side of the Force!
```

[Exercice 03 page suivante](#)

03 - Boucles et listes

1. Créez un fichier `./lib/exercice03.dart`
2. Programmez une boucle **for** permettant d'afficher successivement dans le terminal de commande cette suite de chaîne de caractères.

```
R2-D1  
R2-D2  
R2-D3  
R2-D4  
R2-D5
```

- Créez dynamiquement une liste nommée **droids**, contenant les valeurs de type **String** correspondant au résultat généré précédemment (*R2-D1*, *R2-D2...*, *R2-D5*),
- Affichez la valeur de **droids**,

```
[R2-D1, R2-D2, R2-D3, R2-D4, R2-D5]
```

- A partir de la liste **droids**, créez dynamiquement une nouvelle liste **evenDroids** contenant les valeurs de la liste **droids** dont le dernier caractère est **un nombre pair**. Pour ce faire, employez la méthode **where** de l'objet **List**.
- Affichez la valeur de **evenDroids**.

```
[R2-D2, R2-D4]
```

- Créez dynamiquement une nouvelle liste nommée **reversedDroids**, contenant les chaînes *R2-D5* à *R2-D1*. Pour ce faire, employez la méthode **map** de l'objet **List**.
- Affichez la valeur de **reversedDroids**.

```
R2-D5  
R2-D4  
R2-D3  
R2-D2  
R2-D1
```

[Exercice 04 page suivante](#)

04 - POO + package tiers + tests unitaires

- Créez un fichier `./lib/exercice04.dart`
- Programmez une classe **Starship** disposant des attributs suivants :
 - **id** (String),
 - **name** (String),
 - **cost** (int),
 - **isFlying** (bool) par défaut à **false**,
- Programmez un **constructeur de classe** permettant d'attribuer des valeurs aux attributs **id** (optionnel), **name** (requis) et **cost** (requis).
- Si la valeur de l'**id** est nulle, attribuez automatiquement une valeur de type **UUID v4** à l'attribut **id**. Pour ce faire, importez et employez le package **uuid**.
- Programmez une surcharge de la méthode **toString** afin d'afficher dans le terminal les détails d'un objet de type **Starship**.



```
Starship (id: '4b729b3b-fa69-4814-878c-7a92c0fdbdef', name: 'Star Destroyer', cost: 150000000, isFlying: false)
```

- Programmez une méthode **fly** et **land**, permettant respectivement de faire décoller et atterrir le vaisseau. Modifiez la valeur de l'attribut **isFlying** à **true** lorsque la méthode **fly** est appelée et à **false** lorsque la méthode **land** est appelée.
- Les méthodes **fly** et **land** retournent la valeur **true** si leur exécution n'a pas généré d'erreur.
- Créez une classe **StarshipException** qui hérite de la classe **Exception**.
- Dans le cas où la méthode **fly** est appelée alors que la valeur de l'attribut **isFlying** est à **true**, déclenchez une **StarshipException** avec le message *"Starship is already flying"*.

- Faites de même avec la méthode **land** et déclenchez une **StarshipException** avec le message *"Starship is not flying"* si **isFlying** est à **false**.
- Dans la méthode **run** du fichier, instanciez un objet de type **Starship** et testez les différents scénarios. Employez un bloc **try / catch** pour vérifier les scénarios d'exception sans générer d'interruption du programme.
- Dans le dossier **./test**, créez un fichier **./test/exercice04_test.dart**, importez les classes **Starship** et **StarshipException** puis programmez les **tests unitaires** permettant de vérifier les scénarios nominaux et d'exception pour les cas d'utilisation **fly** et **land**.

Exercice 05 page suivante

05 - Enum, Boucles, random

- Créez un fichier `./lib/exercice05.dart`
- Créez un **enum** nommé **Faction** contenant les valeurs suivantes :
 - *jedi*,
 - *sith*,
 - *resistance*,
 - *empire*,
 - *mandalorian*
- Créez une liste typée (*String*) **names** avec les valeurs suivantes :
 - *Luke Skywalker*,
 - *Leia Organa*,
 - *Han Solo*,
 - *Darth Vader*,
 - *Obi-Wan Kenobi*,
 - *Chewbacca*,
 - *Yoda*,
 - *Lando Calrissian*,
 - *Palpatine*,
 - *Mace Windu*.
- Créez une classe **Character** disposant des attributs suivants :
 - **name** (*String*),
 - **faction** (*Faction*),
 - **force** (*int*),
 - **agility** (*int*),
 - **intelligence** (*int*).

- Les valeurs des attributs de la classe **Character** doivent être **immuables**.
- Pour la classe **Character**, créez un constructeur permettant de disposer de **paramètres nommés** (par opposition aux traditionnels paramètres indexés).
- Dans la classe **Character**, créez un assesseur nommé **power** (*getter*) permettant d'obtenir la somme des 3 capacités (*force, agility, intelligence*).
- Créez une fonction (externe à la classe **Character**) permettant de générer un personnage avec des valeurs aléatoires et à partir de la liste **names** et de l'**enum Faction**.

Pour les attributs force, agility, intelligence, affectez une valeur de 0 à 100.

Ex :

```
Character(name: Darth Vader, faction: resistance, force: 99,
agility: 75, intelligence: 90, power: 264)
```

- Dans la classe **Character**, créez une méthode **capabilityRatios** permettant de calculer et de retourner un objet de type **Map (String, double)** indiquant pour chaque capacité, son ratio par rapport au total des capacités.
- Dans la fonction run,

- utilisez la méthode **generate** de la classe **List** et la fonction **generateRandomCharacter** afin de générer une liste typée contenant 5 objets de type **Character**, nommée **characters**.
- Itérez sur la liste **characters** afin d'afficher chaque personnage, tour à tour, dans le terminal de commande.

```
Character(name: Chewbacca, faction: mandalorian, force:
61, agility: 76, intelligence: 26, power: 163)
```

- En utilisant le résultat retourné par la méthode **capabilityRatios** de chaque objet de type **Character**, affichez dans le terminal de commande, tour à tour, ses trois capacités (*force, agility, intelligence*) sous forme de pourcentage arrondi à deux décimales et suivi du symbole %.

Ex :

```
force: 39.09%
agility: 11.17%
```

intelligence: 49.75%

Exercice 06 page suivante

06 - Fonctions + tests unitaires

- Créez un fichier `./lib/exercice06.dart`
- Importez la classe **Starship** créée précédemment,
- Instanciez une liste de type **Map** nommée **starshipsSelection** contenant les données suivantes : objet *Starship* / quantité sélectionnée.
 - 3 unités de **Star Destroyer** (coût unitaire 150000000),
 - 2 unités de **Sentinel-class landing craft** (coût unitaire 240000),
 - 5 unités de **Y-Wing** (coût unitaire 134999),
 - 8 unités de **X-Wing** (coût unitaire 149999).

```
{
  Starship (name: 'Star Destroyer', cost: 150000000, isFlying: false): 3,
  Starship (name: 'Sentinel-class landing craft', cost: 240000, isFlying: false): 2,
  Starship (name: 'Y-Wing', cost: 134999, isFlying: false): 5,
  Starship (name: 'X-Wing', cost: 149999, isFlying: false): 8
}
```

- Programmez une fonction nommée **calculateCost**, recevant en argument une liste de type **Map<Starship,int>**.
- La fonction **calculateCost** doit retourner la somme de l'attribut **cost** de chaque objet **Starship** contenu dans la liste **Map** passée en argument.

Résultat

451274987

- Dans le dossier `./test`, créez un fichier `./test/exercice05_test.dart`, importez la fonction **calculateCost** puis programmez un **test unitaire** permettant de vérifier son fonctionnement.

07 - Asynchronisme

- Créez un fichier `./lib/exercice07.dart`
- Créez une classe **Vehicle** disposant des attributs suivants :
 - **name** (*String*),
 - **model** (*String*),
 - **manufacturer** (*String*),
 - **costInCredits** (*int*),
 - **length** (*double*),
 - **maxAtmospheringSpeed** (*int*),
 - **passengers** (*int*),
 - **cargoCapacity** (*int*),
 - **starshipClass** (*String*),
 - **url** (*String*).
- Créez une méthode **factory Vehicle.fromJSON** permettant de construire un objet de type **Vehicle** à partir d'un paramètre nommé **json** de type **Map<String, dynamic>** (cf. <https://docs.flutter.dev/cookbook/networking/background-parsing>).
- Ajoutez le package **dio** aux dépendances du projet.
- Créez une classe **VehicleService**.
- Au sein de la classe **VehicleService** programmez une méthode statique **asynchrone** nommée **findAll** retournant une valeur de type **Future<List<Vehicle>>**. Cette méthode doit effectuer une requête **HTTP GET** sur l'**URL** <https://swapi.info/api/vehicles> à l'aide d'une instance de la classe **Dio** puis parser les données au format **JSON** afin de les convertir en objets de type **Vehicle**.
- Programmez une surcharge de la méthode **toString** héritée par défaut, afin d'afficher distinctement un objet de type **Vehicle** dans le terminal à l'aide de l'instruction **print** (cf. <https://www.darttutorial.org/dart-tutorial/dart-method-overriding/>).

Crédits

- Exercice inspiré de la saga *Star Wars* (George Lucas, 1977).
- Données obtenues via l'API *Swapi* (<https://swapi.dev/>).
- Visuel généré à l'aide de <https://fontmeme.com/fr/police-star-wars/>.